

Friday the 13th: Attacking JSON - Alvaro Muñoz & Oleksandr Mirosh - AppSecUSA 2017

Friday the 13th: Attacking JSON - Alvaro Muñoz & Oleksandr Mirosh - AppSecUSA 2017 - OWASP Foundation, YouTube.

- Published: 2017-10-08
- Original: <https://www.youtube.com/watch?v=NqHsaVhlxAQ>
- Preserved from: <https://www.youtube.com/watch?v=NqHsaVhlxAQ> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Friday the 13th: Attacking JSON - Alvaro Muñoz & Oleksandr Mirosh - AppSecUSA 2017

- Video: <https://www.youtube.com/watch?v=NqHsaVhlxAQ>
- Channel: OWASP Foundation
- Published: 2017-10-08
- The media file itself is not archived; this is its metadata and transcript.

Description

Friday the 13th: Attacking JSON

2016 was the year of Java deserialization apocalypse. Although Java Deserialization attacks were known for years, the publication of the Apache Commons Collection Remote Code Execution (RCE from now on) gadget finally brought this forgotten vulnerability to the spotlight and motivated the community to start finding and fixing these issues.

One of the most suggested solutions for avoiding Java deserialization issues was to move away from Java Deserialization altogether and use safer formats such as JSON. In this talk,

we will analyze the most popular JSON parsers in both .NET and Java for potential RCE vectors.

We will demonstrate that RCE is also possible in these libraries and present details about the ones that are vulnerable to RCE by default. We will also discuss common configurations that make other libraries vulnerable.

In addition to focusing on JSON format, we will generalize the attack techniques to other serialization formats. In particular, we will pay close attention to several serialization formats in .NET. These formats have also been known to be vulnerable since 2012 but the lack of known RCE gadgets led some software vendors to not take this issue seriously. We hope this talk will change this. With the intention of bringing the due attention to this vulnerability class in .NET, we will review the known vulnerable formats, present other formats which we found to be vulnerable as well and conclude presenting several gadgets from system libraries that may be used to achieve RCE in a stable way: no memory corruption -- just simple process invocation.

Finally, we will provide recommendations on how to determine if your code is vulnerable, provide remediation advice, and discuss alternative approaches.

Speakers

Oleksandr Mirosh Security Research QA Engineer, HPE Oleksandr Mirosh is working for HPE Software Security Research team investigating and analyzing new threats, vulnerabilities, security weaknesses, new techniques of exploiting security issues and development vulnerability detection, protection and remediation rules.

Alvaro Muñoz Principal Security Researcher, HPE Alvaro Muñoz (@pwntester) works as Principal Software Security Researcher with HPE Security Research (HPSR). His research focuses on different programming languages and web application frameworks searching for vulnerabilities or unsafe uses of APIs.

-

Managed by the official OWASP Media Project
https://www.owasp.org/index.php/OWASP_Media_Project

Transcript

Retrieved with yt-dlp in a container, which is the only route YouTube still answers. Expect transcription errors in type and method names.

so thank you for attending our talk this is Friday the 13th jason attacks we've we have been already introduced but i'm alberto he's Alex he's from Ukraine I'm from Spain so please bear with our strong accents we'll try to best eye to make our best so we have a lot of things to cover today so let's jump straight into it last year was the year of the Java deserialization attacks it was like all over the news and this is weird because this was a very well known attack since at least 2011 and probably the reason that it didn't got into the news like before is because the lack of a good remote code execution gadget in like a popular library like the one that Christopher Hoff and Gabriel Lawrence found in the

Apache Collins collection library and that probably made the community to kind of ignore this moon ability category altogether at least for for Java so at that time the solution by security researchers and consultants were like a stop using Java DC illicit or serialization altogether and developers were like well yeah I would love to do so but I have these objects that I need to store in my database or I need to send across a wire and I need to serialize them somehow so these same consultants and researchers have like okay then use a secure or JSON or XML serialization step we already have some history with finding vulnerabilities remote code execution vulnerabilities in XML satellizer like exit stream or for example X ml decoder so we focus our research on on the JSON serializers so the goal for this talk is twofold first of all check if the adjacent serializers a libraries are any better than the Java and .net CD risers and also to raise some awareness around the dotnet serialization attacks because we saw before that until this remote code execution gadget was found in Java no one was paying attention so what we are going to do is release remote code execution gardius for donuts réaliser so that people start like paying attention to donate and start fixing this kind of vulnerability so the talk is like three different sections we will start like with Jason we will present libraries that are vulnerable to these attacks and gadgets to attack them we will show you some demos on web application frameworks and then we will move completely different section that is dotnet serializers and we will finish with generalizing attacks with to any format any serializer so with that let's jump straight into the JSON section just to set the expectation here we are talking about JSON not like when used by I don't know JavaScript to exchange simple data or JavaScript objects but to replace Java or .net serious realizer which means that we need to serialize Giovannetti objects which means that we need to handle or support some of the object-oriented programming features like I'm going to deal or I'm going to say there lies a java.lang object or how I'm going to serialize a system object in the net I'm going to deal with generics with a polymorphism and so on so a very quick recap on how the Java attacks worked basically the attacker was able to send an untrusted a stream of data which contained both data and also some type discriminators that is nothing else at the class name that should be instantiated by the visualizer so this the serial Isis in this case the object input stream in the Java JRE was basically instantiating an instance of this type that was controlled by the attacker and then invoking some callbacks in this case the read object and read resolve so the attackers were able to our craft gadget change there is nothing else that different classes put it I put together in order to create arbitrary code execution but starting with some of this method of this callback invocation in this case the read object and with resolve however for JSON are we on our the libraries the JSON parsers do not normally invoke this read object and with resolve methods because they have nothing to do with your initialization so we need a different way to start a gadget chain leading to this rhetorical execution so in order to do that who analyze how these libraries parsers reconstruct the objects when they receive the json data and basically they call a default constructor that is the one with no parameter so it's not very interesting from an attacker point of view and then they either use the reflection or setters in order to populate all the fields and properties for the object that they are reconstructing some of them also invoke some a special DC allocation constructor or some callbacks like the Reid object readers read resolved some of them also invoke they donate a specific type

converters that is something that we don't have time to cover in this talk but it's in our paper that was published for the blackhat version of this talk and in addition there are also some common methods that are normally invoke like for example Haas code or equals make it invoke during the DC allocation of a hash map to a string maker invoke do DN when raising an exception for example finalized is always involved by the garbage collector when claiming the memory for that object so there are different methods that are going to be invoked during the D serialization or reconstruction of these objects but by far the ones that are most used are the setters or most common are the setters so we spend some time like looking for gadget chains starting with a setter and leaving two arbitrary code execution to use to attack these libraries so now Alex will show you some of the gadgets that we found let's have a look on a few set regarded in dotnet all of them can lead to code execution some of them have own requirements and limitation but we believe that it's not very difficult to pick up proper one for specific library or specific case the first our regarded is setter of path property in assembler installer type it allows execution of code code execution during lab reloading from path controlled by an attacker there is no additional requirements if assembly this payload is on local machine but in case of resource remote resources and network resources newer dotnet frameworks may have some additional security checks the next two guarded use XML parser and we will show a bit later how it can be used for arbitrary method invocation so settar of property inspector font and color data from workflow designer type requires single-threaded apartment threat it's quite strong requirement but if your target has this configuration you will be able to get remote code execution the next gadget set of source property in research dictionary type has a few requirements to JSON and Marshall it should call setter for types that implement I dictionary interface often in such cases Jason and Marshalls are just populating key value pairs also it should be able to reconstruct system who are high objects and this type doesn't have default constructor but usually a parser can do can do this because they need to work with this type and finally our last gadget it is from object data provider type it's quite flexible and allows a few attract vectors as a result we were able to use it in almost all our own martial arts and formatters let's look on a cold diagram of this gadget setter we'll call it a fresh method that invoke will invoke big inquiry and begin query we'll call quarry worker and finally in invoke method on instance we can see of the vine that will call our arbitrary matter on this slide we can see an example of payload that will pop up calculator for unsafe configuration of JSON net parser the main attack vectors for this gadget are the next we can call non default constructor with own parameters also we can invoke public matters on unmarshal eight objects or we can call any public method including static ones once these own parameters Java has own setter gadget as well after our last year result in G and di injection attacks we have found a few setters with ginger I woke up calls by the way wrestled Oracle disabled era my and Carrabba vectors but aldub vector still works so problem is not solved the first I will graduate its session said session factor G in di name from statistics service class we already mentioned it on our black cat a talk at 2016 the very similar to it is the next gadget to string from a remote client user transaction class and finally set auto commit guarded from already known class JDBC wrote set import this class is from gr e libraries so doesn't require additional dependencies as we can see in this setter we will call connect function and here we can we can notice

initial initial contacts lookup call with our value from data source name properties so they will be able to get remote code execution apart from mansion gadgets that allows code execution by themselves we have a couple other interesting gadget some of them can be used as building blocks for for gadget chains for example Bendix or set data member in dotnet or stink implied to string in Java can be used for arbitrary gutter calls so we can use other gadgets in gutters to get code execution other gadget can be used for non our C attacks for example set in XML in XML data document or said set data view setting collection string from that of you manager type can be used for XML external entities attack in some version on that net from framework now Alvaro will show where we can use all these gadgets so we analyzed our like a bunch of different J's libraries for both Java and .net and we like figured out these three high-level requirements for a library to be vulnerable so first of all are the library needs or the attacker needs to be able to control the type that is going to be instantiated by the library so that means that in the JSON data you will normally see like some attributes like for example with this underscore type dollar type class name Java class name some attributes which contain a value that looks like Java a namespace or dotnet assembly and then those values such as strings are going to be used by the library to instantiate those types in the server then the library is going to invoke some methods on those reconstructed objects we already saw that for example finalized we'll always get invoke and that most libraries will invoke setters and last but not least the attacker needs to be able to have like a large enough graduate space for those methods to be able to craft these gadget chains leading to arbitrary code execution so we categorize all the libraries that we analyzed and we like use two different factors for this categorization the first one is whether these libraries are including the type discriminator that that is the type name by default in the in the data or the developers need to enable some configuration setting for the library to include and process this type information and the second factor is how do they protect against some kind of type attacks so how do they control the types so some of the libraries performs just a cast operation after the DC allocation has completed which means that by that time the payload has already been executed and when you get the cast exception is already too late and some of them performs what we call the inspection of the expected type object graph which sounds like a little bit complex but it's nothing else that taking the object graph of what I'm expecting like for example I am expecting a user instance or our cart instance then I'm going to inspect the object graph and for each of the fields or properties when they say arrives in the data from the from the JSON data I'm basically going to check the Sigma bility for what I get to what I'm expecting so if I'm expecting for example I'm the CIC in the property the name property for the user I'm expecting a system string and I'm not supposed to get an object data provided that is the cat that I saw you before so because object that I provided is not a signal to string I will raise an exception and will not process to visualize that object that are provided there are other libraries that in addition to that they create like a wide list of allowed types at construction time which means that they inspect the object graph then they take all the types that they can find in the object graph and then they build wide lists with those types so at one time time when the serializing untrusted data they applied that whitelist and say okay I'm expecting these five types and you're sending me some type that is not in my whitelist so I'm not going to continue with this we found that this is

still vulnerable if attackers can either control the expected type that you may think that is not common but we will show you some real-world examples in very popular CMS and also the attacker can find an injection point in the object graph that is a signal to their payload so let's say for example that we have this object graph here oops sorry okay so imagine that we have the subject graph here and any properties in red are those that are a signal to for example any on any other type like for example in dotnet the hash table or array lists are generic collections so they contain items that can be asking about to any type or if you have like a dictionary of string objects you can place your payload there or maybe you have like for example an exception field or property but attackers can up use any exception so they cannot say for example use the validation exception which contains a value property that is a system object so they can inject the payload there so with that this is the summary of the libraries that we analyzed the ones in red is are the ones that are vulnerable by default so they should not be used with untrusted data basically they include the type information by default and then they perform just a cast operation so they perform no type control at all then they invoke setters as you can use this setter gadgets that we presented in order to get arbitrary code execution then we have the yellow and orange ones which are not insecure by default but depending on how the Vela pairs use them they may they may make them vulnerable so but normally they don't include the type information by default and they perform some of this inspection of expected type object graph and you may think that finding an application that is using this these libraries and meeting or satisfying both requirement is difficult but it's kind of the other way around because if the developers for example contains a system object property in their object graph they will be forced to include the type information in the JSON data otherwise they won't be able to serialize it and if you find an application that is including this type information it's probably because they have some system object or some generic collection I am properties in their objective then we have the one in in green that is this one we put it in green not because it's completely secured you can still implement a type of up there that is bootable but I mean you have to do it like in purpose and in addition they don't invoke any methods they don't invoke setters so the only entry point for you is finalized which is kind of in the gutted space so these are some of the libraries that we serve for example file system was one of the ones in red as I said should never be used with untrusted data we've prom for example a CMS called calico CMS that was taken and trusted data and I use in translation to digitize it you should never do that JavaScript serialize is one of the dotnet native JSON serializers by default it does not include the type information so it's secure however developers can initiate it by passing a type resolver which means that the library will include and process the type information and because it does just post this realization cast operation as the type control it will be vulnerable and you can it cannot be used with untrusted data when used with type resolvers then we have the data contract json serial i said that is another of the dotnet native JSON libraries and this is probably one of the most secure because it's limited it cannot serialize any type is quite limited but it's a it's applying this technique of creating a whitelist based on the object graph types and then apply in that whitelist at runtime time but we found like we said before that it's still vulnerable if attacker can control the expected type so here's an example from from github where you can see that add expected type is controlled because

it's actually coming from a cookie and you will see that in an example that we showed you before in a demo so last but not least JSON Dannette is probably the most popular JSON library for .NET it's even recommended by Microsoft over their own JavaScript serializer or the data contract agency realised by default it's secure because it does not include the type information but again developers can enable this type name handling setting in order to include this type information and if that is the case and the attacker can find an entry point it will be vulnerable so this is another example from github we have this message class here it contains just one property but this property happens to be a system object property which means that in order to be serialized as we said before the developers need to include the type information in there along with the data in the JSON data because otherwise it's not going to be possible to be serializable and that's why they use this JSON property annotation here which is configured to include the type information so let's show you some examples and so for example we have this demo for breeze that is a .NET framework to build RESTful API is for the data management and data management so basically do you expose a REST endpoints to perform operations on your entities like for example add a new record update a record at the little record and so on and they offer both clients for both .NET and JavaScript so they use this type name this global JS materializer which is configured as you can see lines 56 to include the type information when citizen and deserializing system object properties then at a different point it's visualizing untrusted data and as you can see line 63 the expected type in this case is safe options so now we have to do this inspection of the object graph expected type which is nothing else than taking a look into the safe options type spec the object graph it's just two properties and see that the second one is a system object so it's going to be a signal to any type so we are going to be able to inject our payload here so let's see if I can play this without breaking PowerPoint okay so this is an application that comes with a breeze is one of the sample applications for developers to learn the framework and as you can see here if you change the model the model name of one of the cars well it's just sending a request to the endpoint to update one of the records now if we capture the request you can see what we're expecting so that for example they change a name but we also see an empty safe options dictionary now if we send this request and we analyze the response we will see that there is this dollar type attribute that contains something that looks like .NET class name so that's because it's using the Internet and it's including tab information so now we are going to use our object data provider gadget in order to invoke the process .NET start method and pop up a calculator when DC realizing the data so now we're going to replace the empty safe options dictionary with our payload and we are basically send that to the server that is not running any calculators right now and as soon as we send it we will be able to execute any process in the server so we will get remote code execution thank you so this is a web application frameworks meaning that any application built using breeze was vulnerable to these to this attack so a big shout-out to a married bachelor who published a similar research to ours who says the kind of the same premises and conclusion conclusion but he focused exclusively on Java and here were left with our research on both the Jackson and Jason i/o library so the for the JSON i/o he found a completely different approach so congrats there and also he overlaps in our daily BC roasted implementation gadget which was kind of an obvious choice because was used

previously on the jela visualization world but he also found a very interesting gadget so we highly recommend you to go and visit the Marcell sick white paper that contains a lot of interesting stuff around these kind of attacks so now we will switch gears and move into the dotnet part in dotnet world they put potential security problems with binary format and data contract either were known for the years the great work of James for show about weaknesses and attack vectors from main dotnet formatters was presented at blackhat 2012 so more than five years ago anyway we could not find public publicly available good remote code execution guard chain there was one published by florell florian gold here but it used memory corruption so it's not very suitable for stable exploit but we were sure that there should be a lot for code execution during dotnet and deserialization so we have spent some our time for searching such a gadget and have found found one that can be used in binary and some other dotnet for matters but after our our work was ready and accepted by blackhat at the same James partial published a couple new RC gadgets as part of his own research by the way not connected with our topic and anyway you can find the tails above about here this gadget on his a post in Google project zero block here we will focus on our own so we will use PS object type it is part of PowerShell libraries so is available on almost all Windows machines and before we go further there are couple remarks about this gadget in PowerShell version one and this type is not serializable so it's not suitable for attack but all modern version of Windows static from Windows 8 and Windows Server 2012 are shipped is newer and vulnerable version the next remark may be reported this issue to Microsoft and they released a fix so if you are not ignoring their updates you should be safe so Pierce object uses custom the sterilizer and in case of tea I am instance it will call ray his raid see I am instance method and we can see here that attacker is able to specify own type as element type of array and the sterilizer will try to reconstruct object of this type to find a proper way to do this the cerrado will use figure conversion method it's quite interesting method for attacker as there a lot of ways for for attacks we highlighted only the most obvious one so we can call non default constructor with one argument and we can control its value also we can invoke setters on public properties so we can use here our mentioned gadget and finally we can call a static public parse method on arbitrary type let's try to use in this case this one so as we said earlier exam reader parse can be used for arbitrary method invocation here we can see payload that we'll call process start method with our string as an argument we can not just hear our namespace assembly type method name and finally calc as an argument so we can use this pilot for start any process apart from mentioned binary and a data contract serializer dotnet offers a lot of other formats that can be addressed unfortunately we don't have enough time for the Teletubby of each of them so we will try to briefly cover them in our talk and if you are interested in deeper analysis we can offer out white paper as a good source so we can group all this dotnet for matters in two big groups formats that are vulnerable in the full configuration like binary format so up for matter and nab data contract serializer also we can add here other for matters that internally use them like loss formatter object state format binary message for matter they should not be used with untrusted data or you have to proper configure them to limit available types for example use type binder with white listed types the second group is is formatter that are safe in the full configuration like XML sterilizer data contract serializer data contract Jason sterilizer

but if you are using the weak data contractor is over by the way we have met examples even on official Microsoft documentation or an attacker is able to control expected type you will have very serious security problem and we will show a bit later that in this case code execution is real even for the most limited for matters for example like XML serialize now rescission back to Alvaro he will show next demo yeah so because are the most worth 4,000 slides let's see if I can play this so this is a Nancy it's a web application framework for dotnet it's basically very similar to the Sinatra framework for Ruby probably that's why it's called Nancy and well they care about security so they have protections against it serve which is good but the way they implemented it is instead of putting like a unique token into the CSR cookie directly they put that in as the property of a special class and then they see realize this class basics t4 encoded and put that into the cookie so the result is that they have this big basis takfir encoded blob starting with this eh-eh-eh-eh d that by the way is the magic number which means that you cut remote code execution and now because it's using binary format we can use our PS object card yet to get arbitrary code execution when this cookie is dis I realized so let's see that in action this is one application build using the Nancy framework if we take the cookies we will see that it contains these basics different coded NC serve cookie which start with our magic number which is good and then if we check the source code we will see that it's double submitting the same value as part of a hidden form field so that's good double submission is better than single submission and now if we replace the value of the cookie with the PS object cut yet that is basically configured to run the calculator we will just paste it as the value of the of the NCR cookie who will fill the form and we will send it to the server so the framework when deserializing the end Souths cooking will basically DC realize how we're entrusted payload and will run the calculator so we get an exception here but we have our calculator so we were able to attacked any application build using this in NC framework so last the last section is generalizing the attack we saw that this attack is not really specific to JSON we don't see any JSON stuff especially this when a specific stop being used so there is nothing really specific to either JSON or the binary for matter for the net or for the XML see realized the problem here is that BCC Eliezer needs to reconstruct the object and for that if the attacker can control the type then they are going to be able to initiate some call graph that they can control and may lead to arbitrary code execution so again these are the three simple requirements that we saw you before attacker needs to be able to control the type which means that they type discriminator need to be included in the sterilized format some methods needs to be invoke on the reconstructed object which is normally always the case because the finalize method will normally be invoke and also there should be a gadget or a large enough catheter space for the attackers to craft a gadget chain leading to arbitrary code execution and starting with these invoke methods the truth is that most of the library's most of the serial Isis will invoke the setter so the caches that we presented here today the object data provided for dotnet and the JDBC rotating pole for Java are going to be valid gadgets for attacking most of other libraries so some example we found like many of them but we don't have time our notice space to present all of them for example FS Pickler or sub sterilizer are some dotnet libraries that not only work with dot net framework but also it's Silverlight Windows Mobile and other frameworks and they both include the type information by default one of the EM performs no type control is just

doing a post this realization cast and the other one FS piglet is doing this object graph inspection which means that you need to still find an entry point but both both of them should not be used with untrusted data we have wire that is now known as Hyperion this is the CLI sir for a cathode net that is basically the dotnet version of the famous Scala and Java library to do some actor based concurrency and basically we have this actors that are exchanged in messages that used to be serialize with JSON dotnet using the unsafe configuration by the way that and the type name handling equals oh so including the type information and now they replace that with this Hyperion sterilized that is a custom sterilizer which is basically including type information invoking setters and also invoke in some other callback so it's more honorable also be aware of role in your own format when we notify Nancy about their vulnerability they said that they were using a different format in their pre-release two attacks branch and we took a look at the new format which was basically custom JSON format that they implemented on their own instead of using I don't know Jason dotnet or some existing libraries so we took a look at this custom format and we saw that they were using a type the type discriminator they were including the type in a discriminator and then invoke in the setter so it was also vulnerable to remote code execution then we have done net nuke CMS that is one of the most important most popular CMS for for dotnet which is kind of wrapping xml said Eliza so instead of just using xml sheila serializer would have fixed that type is basically sending the expected type and then the payload for the XML serialization this is this was kind of a challenge for us because XML satellizer as Alex said before is the most limited serialize ER in the .net family so if we could get a rhetorical execution here we were kind of confident that we were going to be able to get a bit Oracle execution for any of the other sterilizers so we try to use our object data provider gadget that was XML serialization resource so far so good but then we could not use the process to invoke the process dot star because process is a type which contains interface members that is one of the limitations for this serializer that was not a big problem because we can use other payloads like for example invoke the summer reader load method or object the state formatter deserialize method or you can even look in the target application class path for custom gadgets to use so for example dotnet new contain this file system unix class which is kind of handy for hackers because it contains some methods that can be used to deploy web cells or to read arbitrary files into the HTTP response and send their contents back to the attacker so that problem was solved and then we have another problem basically XML sterilizer used this whitelist approach is building this whitelist at construction time that then is applying at runtime time and if we analyze the object graph of the object data provider type we will find one system object property which it will be included in the whitelist but at runtime we will be sending something else we will be sending the file system doodles or the sam'l reader or something else so it won't be in the whitelist and then it will make the disc allocation to fail so we found this clever trick of using a parameterized type and put in our runtime types in the past the parameterized types for the payload and then that makes XML serialization earn about our runtime types at construction time I don't know if that makes any sense anyway with that we were able to get a botanical execution remotely on any instance of DotNetNuke and basically that was some kind of simple so we have this DNN personalization cookie that is read at some point for example when visiting the 404 error

page this piece of code is executed and then this is reading the the cookie that contains an XML value and then is passing that XML to the deserialize hash table XML method then in this method we are basically extracting the type of the expected type from the XML and using that to instantiate a XML serialization instance in line 201 which this basically means that we now control the expected type so that's what we were talking before that is something that is kind of common and then at the rest of the cookie the rest of the XML is DC realized using this this XML visualizer so this is how a regular cookie looks like basically I'm saying okay I'm sending a system boolean and this is the system boolean that I'm sending serialized with XML serializer this is our payload which basically say I'm sending this and by the way include these types into the whitelist they're the filesystem you do this one for example and this the inner box is the payload that is going to be dis a realized by XML serialization as you can see here we are using the file system utils class in the internet nuke class path to use a de poule file method to download aware web cell and deploy it on the target server so with that let's see these plays so this is a DotNetNuke out-of-the-box installation if we visit a non-existent page we will get the 404 error page which we know that it's going to read this dnm personalization cookie so now if we intercept the request with any proxy include this dnm personalization cookie like for example let's use verb and then the cook is not present here but we know because we have so the source code that is going to be processed so let's put it here and we will basically copy paste a piece of åkesson of XML that I showed you before that is basically using the filesystem uriel stab the pool file method to deploy to download on the ploidy and the web channel so let's copy paste this here let's format it a little bit and now if we check in the server side the one that is running there the CMS we will see that there is no cell aspx file and now as soon as we send a request and this gets this realized we will get our CL cell aspx deployed into the server and ready for use by the attacker so now we only have to go and visit this cell aspx get our web sell and then we can interact with the with the server and running commands thank you so with that we're running out of time so to quickly wrap up do not use any do not DC lies and trust the data we saw that is this is Noah programming Jerrod visualization is not a problem in binary formatted internet or JSON dotnet it's basically a problem in disguising and Trust the data with time controlled by the attacker so try to not do that I need you have to try to evaluate the security of the layout of the library that you are using avoid libraries that are performing just cast operations and not a pass trick type inspection of the expected type including the type discriminator the type names is necessary but not sufficient but if you find a steriliser format that includes the type discriminators that's a good indicator that is going to be vulnerable and also don't use information or data controlled by the attacker as the expected type because that means basically that they can control the object graph and so they came over and last but not least do not roll your own format if you're not aware of these kind of attacks and don't know how these things can can break so with that thank you very much these are our email address if you have any questions and if you have any questions you can take it here [Applause]